WalletPlatform

Low Level Design Document

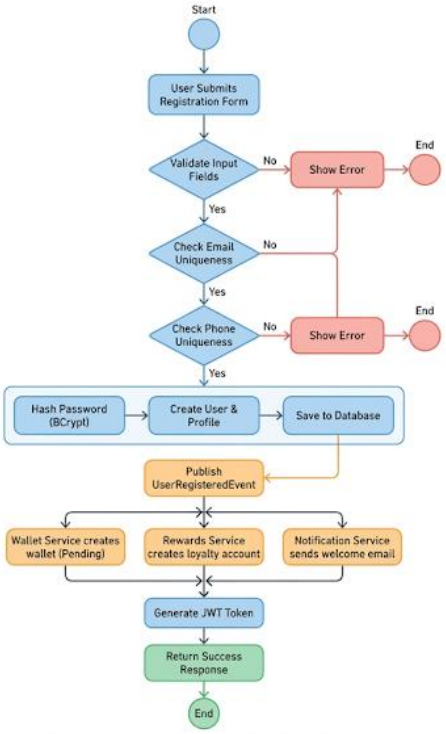
Microservices Wallet & Payment Platform

Document Type	Low Level Design (LLD)
Covers	Flow · ER · Architecture · Class · Schema diagrams
Services	8 microservices + shared library + gateway
Version	1.0 — Final

1. Flow Diagrams

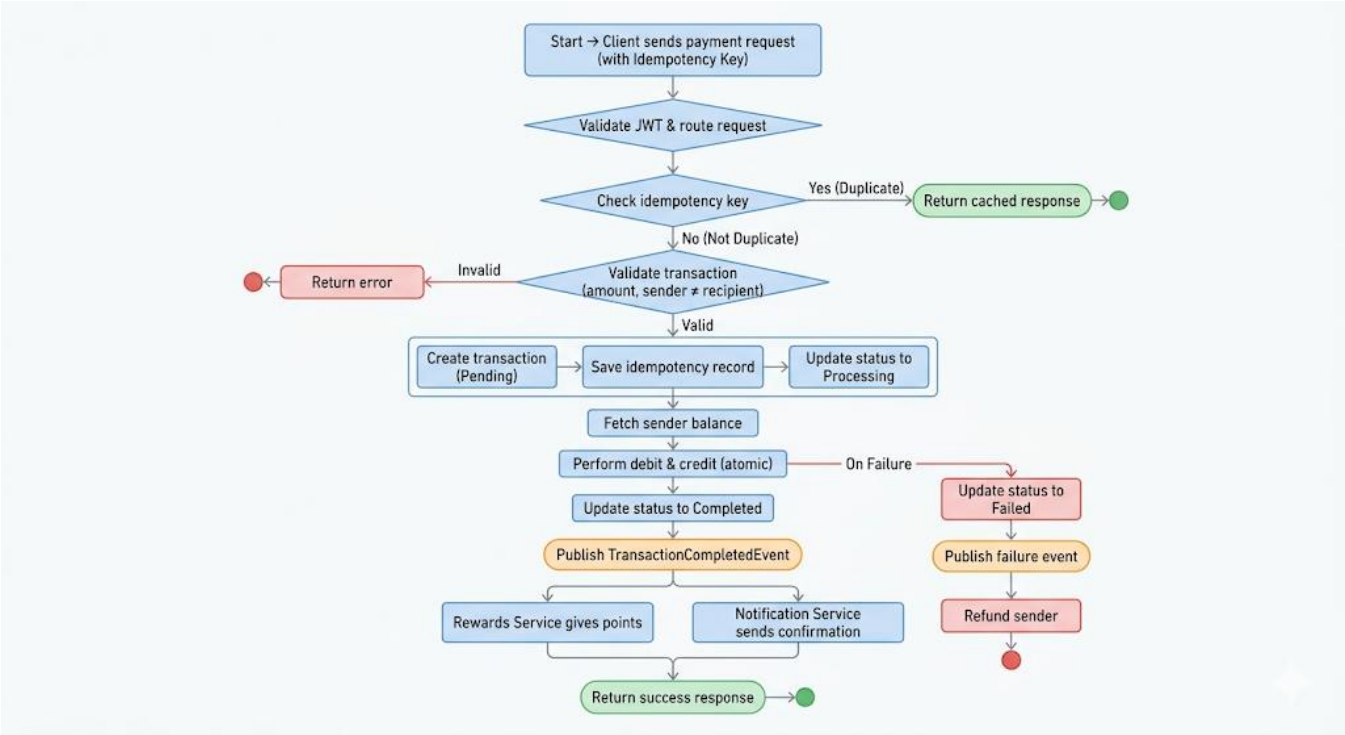
1.1 User Registration Flow

The registration flow validates input, checks for duplicates, hashes the password, persists the user, and publishes a UserRegisteredEvent. Three services react asynchronously.

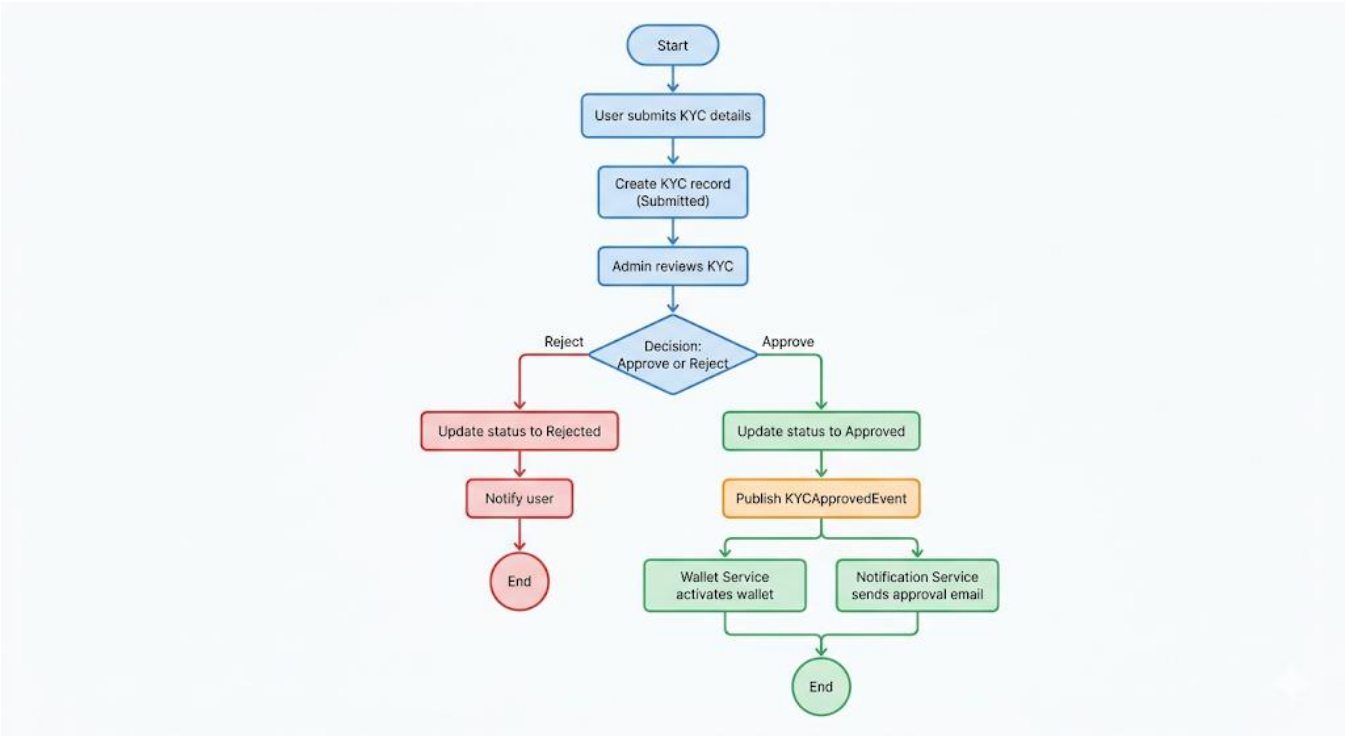


1.2 Payment Transaction Flow

The payment flow is the most complex in the system. It uses idempotency for safety, double-entry accounting for integrity, and Saga compensation for failure recovery.



1.3 KYC Approval Flow



2. Database Schema

2.1 auth_db

Users table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Primary key — Guid.NewGuid() in C#, non-sequential for security
Email	NVARCHAR(256)	UNIQUE, NOT NULL	Stored lowercase, unique index for fast login lookup
PasswordHash	NVARCHAR(MAX)	NOT NULL	BCrypt hash — never the plain password
PhoneNumber	NVARCHAR(20)	UNIQUE, NOT NULL	Stored with country code, unique index
Role	NVARCHAR(50)	NOT NULL	Stored as string: User / Admin / Agent
IsActive	BIT	NOT NULL, DEF 1	Soft delete — false means deactivated, data retained
IsEmailVerified	BIT	NOT NULL, DEF 0	Email verification flag — future use
CreatedAt	DATETIME2	NOT NULL	UTC timestamp set by C# code
UpdatedAt	DATETIME2	NULL	Null until first update

UserProfiles table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Own primary key
UserId	UNIQUEIDENTIFIER	FK → Users, CASCADE	One-to-one with Users, cascades on delete
FirstName	NVARCHAR(100)	NOT NULL	
LastName	NVARCHAR(100)	NOT NULL	
AvatarUrl	NVARCHAR(MAX)	NULL	Profile picture URL
DateOfBirth	DATE	NULL	Optional — for age verification
Address	NVARCHAR(MAX)	NULL	
City	NVARCHAR(MAX)	NULL	
State	NVARCHAR(MAX)	NULL	

Column	Type	Constraints	Description
PinCode	NVARCHAR(MAX)	NULL	

KYCRecords table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
UserId	UNIQUEIDENTIFIER	FK → Users, UNIQUE	One-to-one — one KYC per user
DocumentType	NVARCHAR(50)	NOT NULL	Aadhaar / PAN / Passport
DocumentNumber	NVARCHAR(100)	NOT NULL	String to handle alphanumeric doc numbers
DocumentUrl	NVARCHAR(MAX)	NULL	Secure storage URL
Status	NVARCHAR(50)	NOT NULL	Pending / Submitted / Approved / Rejected
RejectionReason	NVARCHAR(MAX)	NULL	Only populated on rejection
ReviewedBy	UNIQUEIDENTIFIER	NULL	Admin user ID — null until reviewed
SubmittedAt	DATETIME2	NOT NULL	UTC timestamp
ReviewedAt	DATETIME2	NULL	Null until admin action

RefreshTokens table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
UserId	UNIQUEIDENTIFIER	FK → Users, CASCADE	Many refresh tokens per user
Token	NVARCHAR(512)	UNIQUE, NOT NULL	128-bit random, base64 encoded
ExpiresAt	DATETIME2	NOT NULL	7 days from creation
IsRevoked	BIT	NOT NULL, DEF 0	Set true on logout or rotation
ReplacedBy	NVARCHAR(512)	NULL	New token after rotation — audit chain
CreatedByIp	NVARCHAR(MAX)	NULL	IP for security monitoring
CreatedAt	DATETIME2	NOT NULL	

AuditLogs table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
UserId	UNIQUEIDENTIFIER	NULL	Null for failed logins with unknown user
Action	NVARCHAR(100)	NOT NULL	REGISTER / LOGIN / LOGIN_FAILED / KYC_SUBMIT
Resource	NVARCHAR(200)	NOT NULL	Service name — Auth / KYC
IpAddress	NVARCHAR(MAX)	NULL	Client IP from X-Forwarded-For or RemoteIpAddress
UserAgent	NVARCHAR(MAX)	NULL	Browser/client identifier
Success	BIT	NOT NULL	True/false outcome
Details	NVARCHAR(MAX)	NULL	Additional context for investigation
CreatedAt	DATETIME2	NOT NULL	UTC — indexed for time-range queries

2.2 wallet_db

Wallets table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
UserId	UNIQUEIDENTIFIER	UNIQUE, NOT NULL	Denormalized FK — no FK constraint to auth_db
WalletNumber	NVARCHAR(20)	UNIQUE, NOT NULL	WLT + timestamp + random — human readable
Balance	DECIMAL(18,2)	NOT NULL, DEF 0	Total balance including frozen amount
FrozenAmount	DECIMAL(18,2)	NOT NULL, DEF 0	Amount held in escrow for pending txns
Currency	NVARCHAR(3)	NOT NULL, DEF INR	ISO currency code
Status	NVARCHAR(50)	NOT NULL	Pending / Active / Frozen / Closed
CreatedAt	DATETIME2	NOT NULL	
UpdatedAt	DATETIME2	NULL	

WalletFreezeLogs table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
WalletId	UNIQUEIDENTIFIER	FK → Wallets	
ActionBy	UNIQUEIDENTIFIER	NOT NULL	Admin user ID who performed the action
Action	NVARCHAR(20)	NOT NULL	FREEZE or UNFREEZE
Reason	NVARCHAR(500)	NOT NULL	Mandatory reason for audit purposes
CreatedAt	DATETIME2	NOT NULL	

2.3 ledger_db

Transactions table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
SenderId	UNIQUEIDENTIFIER	NOT NULL, IDX	Indexed for user transaction history queries
RecipientId	UNIQUEIDENTIFIER	NOT NULL, IDX	Indexed for received payments queries
Amount	DECIMAL(18,2)	NOT NULL	Decimal for exact monetary arithmetic
Currency	NVARCHAR(3)	NOT NULL, DEF INR	
Type	NVARCHAR(50)	NOT NULL	WalletFund / PeerTransfer / MerchantPay / Refund
Status	NVARCHAR(50)	NOT NULL	Pending / Processing / Completed / Failed / Reversed
Description	NVARCHAR(500)	NULL	User-provided payment description
IdempotencyKey	NVARCHAR(128)	UNIQUE, NOT NULL	Client-supplied key — prevents duplicate processing
FailureReason	NVARCHAR(MAX)	NULL	Populated on Failed or Reversed status
ReferenceId	NVARCHAR(MAX)	NULL	External reference (merchant order ID etc.)
CreatedAt	DATETIME2	NOT NULL, IDX	Indexed for date-range reporting
CompletedAt	DATETIME2	NULL	Set when status → Completed
ReversedAt	DATETIME2	NULL	Set when Saga compensation applied

LedgerEntries table — IMMUTABLE

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
TransactionId	UNIQUEIDENTIFIER	FK → Txns, RESTRICT	RESTRICT — cannot delete transaction with entries
AccountId	UNIQUEIDENTIFIER	NOT NULL, IDX	UserId acting as account identifier
EntryType	NVARCHAR(50)	NOT NULL	Debit or Credit
Amount	DECIMAL(18,2)	NOT NULL	Always positive — EntryType determines direction
Currency	NVARCHAR(3)	NOT NULL	
BalanceBefore	DECIMAL(18,2)	NOT NULL	Snapshot — balance before this entry
BalanceAfter	DECIMAL(18,2)	NOT NULL	Snapshot — balance after this entry
Description	NVARCHAR(500)	NULL	Human-readable description of the entry
CreatedAt	DATETIME2	NOT NULL, IDX	

IdempotencyKeys table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
IdempotencyKey	NVARCHAR(128)	UNIQUE + UserId	Composite unique: same key allowed for different users
UserId	UNIQUEIDENTIFIER	NOT NULL	Scopes idempotency to the requesting user
TransactionId	UNIQUEIDENTIFIER	NOT NULL	Points to the original transaction for cached response
ResponseJson	NVARCHAR(MAX)	NULL	Cached serialized response
CreatedAt	DATETIME2	NOT NULL	
ExpiresAt	DATETIME2	NOT NULL	24 hours after creation — enables cleanup job

2.4 rewards_db

RewardTiers table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Fixed GUIDs in seed data — immutable across migrations
Name	NVARCHAR(50)	UNIQUE	Bronze / Silver / Gold / Platinum
MinPoints	INT	NOT NULL	Inclusive lower bound for this tier
MaxPoints	INT	NOT NULL	-1 = unlimited (Platinum tier sentinel value)
MultiplierFactor	DECIMAL(5,2)	NOT NULL	1.0 / 1.5 / 2.0 / 3.0 — applied to raw points calc
BadgeColor	NVARCHAR(20)	NOT NULL	Hex color for UI display
DisplayOrder	INT	NOT NULL	Sort order for tier display: 1=Bronze, 4=Platinum
IsActive	BIT	NOT NULL	Allows disabling tiers without deleting

PointRules table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
Name	NVARCHAR(100)	NOT NULL	Human-readable rule name
TransactionType	NVARCHAR(50)	NOT NULL	PeerTransfer / MerchantPay — determines when rule applies
PointsPerRupee	DECIMAL(10,4)	NOT NULL	e.g. 0.1 = 1 point per ₹10 spent
MinAmount	DECIMAL(18,2)	NULL	Transaction must be >= this to qualify
MaxAmount	DECIMAL(18,2)	NULL	Cap the amount used in points calculation
MaxPointsPerTxn	INT	NULL	Absolute ceiling on points per transaction
IsActive	BIT	NOT NULL	Allows disabling rules without deleting

LoyaltyAccounts table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
UserId	UNIQUEIDENTIFIER	UNIQUE, NOT NULL	One loyalty account per user

Column	Type	Constraints	Description
TierId	UNIQUEIDENTIFIER	FK → RewardTiers	Current tier — updated on threshold crossing
TotalPoints	INT	NOT NULL, DEF 0	Current spendable balance — decreases on redemption
LifetimePoints	INT	NOT NULL, DEF 0	Cumulative earned — NEVER decreases, drives tier calc
RedeemedPoints	INT	NOT NULL, DEF 0	Total points ever redeemed — for analytics
CreatedAt	DATETIME2	NOT NULL	
UpdatedAt	DATETIME2	NULL	

PointTransactions table — IMMUTABLE

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
LoyaltyAccountId	UNIQUEIDENTIFIER	FK → LoyaltyAccts	
Type	NVARCHAR(50)	NOT NULL	Earned / Redeemed / Expired / Adjusted
Points	INT	NOT NULL	Positive = earned, Negative = redeemed
BalanceBefore	INT	NOT NULL	Snapshot — for audit and dispute resolution
BalanceAfter	INT	NOT NULL	Snapshot — for audit and dispute resolution
Description	NVARCHAR(300)	NOT NULL	Human-readable e.g. 'Points from PeerTransfer of ₹500'
ReferenceId	UNIQUEIDENTIFIER	NULL, IDX	TransactionId or RedemptionId for traceability
CreatedAt	DATETIME2	NOT NULL, IDX	

2.5 catalog_db

CatalogItems table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Fixed GUIDs for seeded items
Name	NVARCHAR(200)	NOT NULL	Display name of the reward item
Description	NVARCHAR(1000)	NULL	
Category	NVARCHAR(50)	NOT NULL, IDX	Voucher / Cashback / Product / Subscription / Donation
PointsCost	INT	NOT NULL	Always a whole number of points
ImageUrl	NVARCHAR(MAX)	NULL	CDN URL for item image
Brand	NVARCHAR(100)	NULL	Brand name for display
StockCount	INT	NOT NULL	-1 = unlimited, 0 = out of stock, N = limited stock
IsActive	BIT	NOT NULL, IDX	Soft disable without deleting catalog history
ValidFrom	DATETIME2	NOT NULL	Item available from this date
ValidUntil	DATETIME2	NULL	Offer expiry — null = no expiry

Redemptions table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
UserId	UNIQUEIDENTIFIER	NOT NULL, IDX	Denormalized ref — no FK to auth_db
CatalogItemId	UNIQUEIDENTIFIER	FK → CatalogItems	RESTRICT — cannot delete item with redemptions
PointsSpent	INT	NOT NULL	Snapshot of cost at time of redemption
Status	NVARCHAR(50)	NOT NULL, IDX	Pending / Processing / Completed / Failed / Cancelled
VoucherCode	NVARCHAR(100)	NULL	Generated on Completed — format: XXXX-XXXX-XXXX
FailureReason	NVARCHAR(500)	NULL	Populated on failure
CreatedAt	DATETIME2	NOT NULL	
CompletedAt	DATETIME2	NULL	Set when voucher generated

2.6 notification_db

NotificationTemplates table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
Type	NVARCHAR(50)	UNIQUE(Type+Channel)	Welcome / TransactionSuccess / PointsAwarded etc.
Channel	NVARCHAR(50)	UNIQUE(Type+Channel)	Email / SMS / Push / InApp
Subject	NVARCHAR(300)	NOT NULL	Email subject — supports {Placeholders}
Body	NVARCHAR(MAX)	NOT NULL	Message body — supports {FullName}, {Amount} etc.
IsActive	BIT	NOT NULL	Disable template without deleting

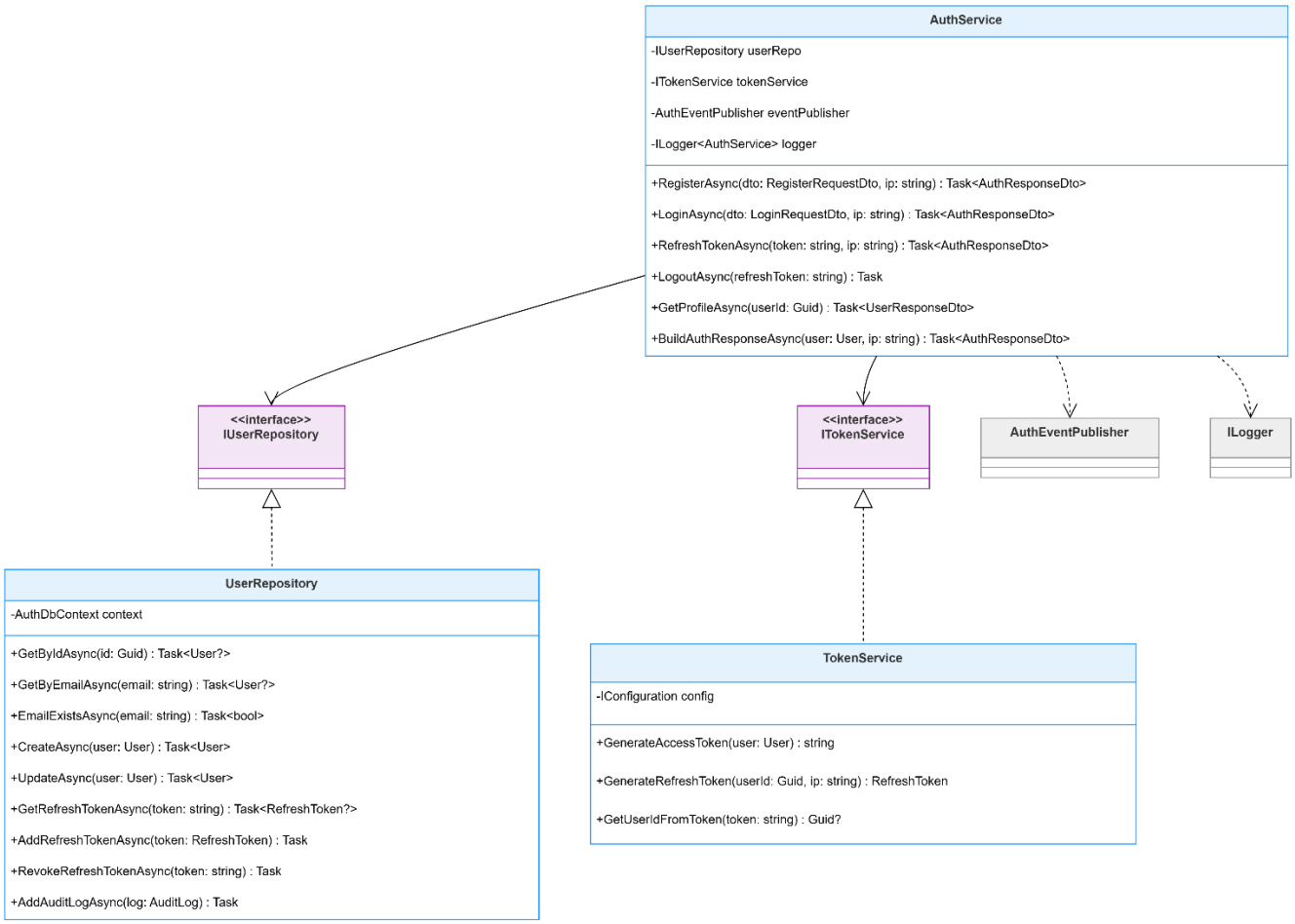
NotificationLogs table

Column	Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	
UserId	UNIQUEIDENTIFIER	NOT NULL, IDX	
Type	NVARCHAR(50)	NOT NULL, IDX	Notification type — for analytics
Channel	NVARCHAR(50)	NOT NULL	Email / SMS
Status	NVARCHAR(50)	NOT NULL	Pending / Sent / Failed / Skipped
Recipient	NVARCHAR(256)	NOT NULL	Email address or phone number
Subject	NVARCHAR(300)	NULL	After placeholder substitution
Body	NVARCHAR(MAX)	NULL	Full rendered message body
FailureReason	NVARCHAR(500)	NULL	Provider error message
ExternalRef	NVARCHAR(200)	NULL	Provider message ID for tracking
CreatedAt	DATETIME2	NOT NULL, IDX	
SentAt	DATETIME2	NULL	Populated on successful delivery

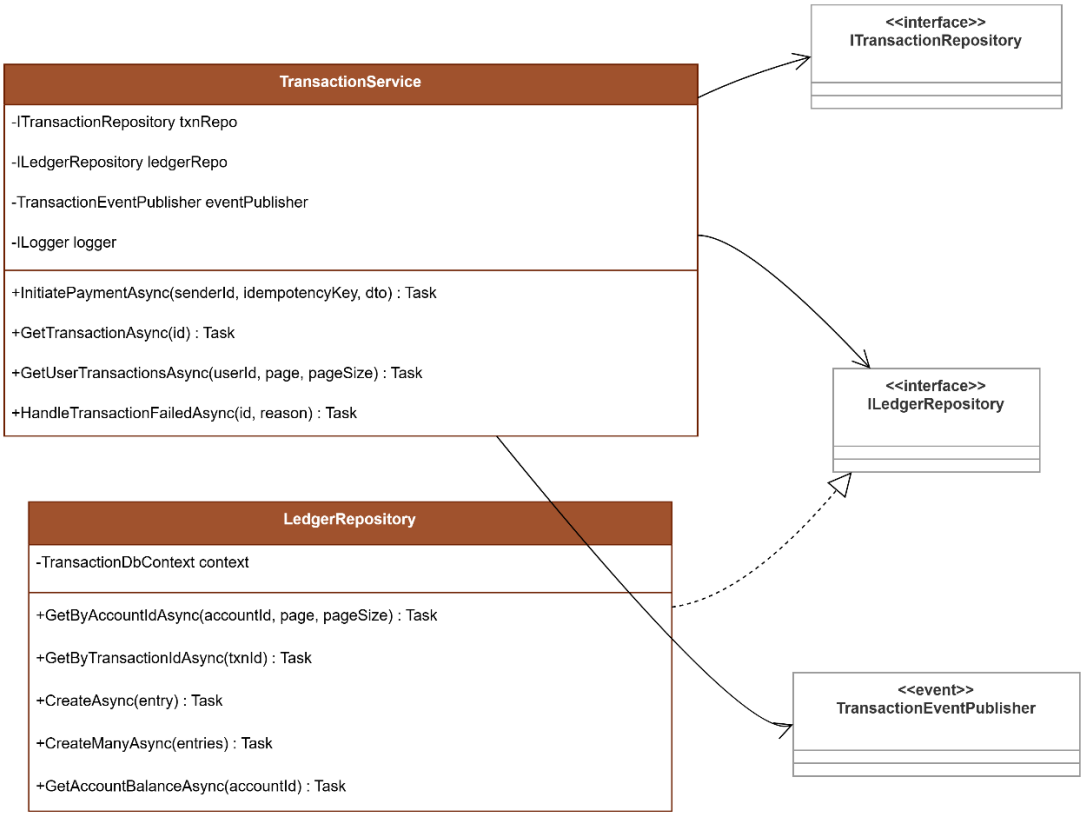
3. Class Diagrams

Class diagrams show the relationships and responsibilities within each service layer. Only the most architecturally significant classes are shown.

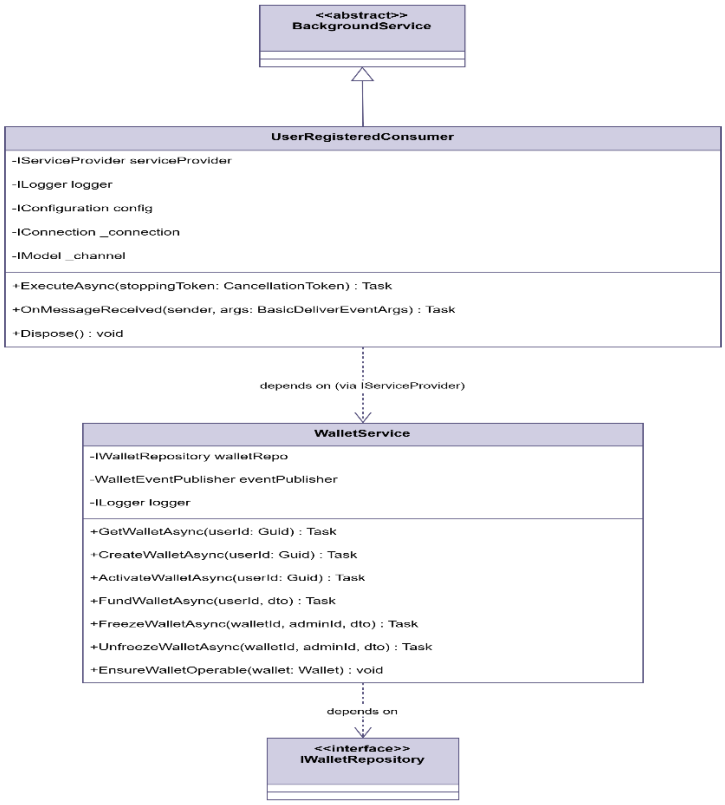
3.1 Auth.API — Core Classes



3.2 Transaction.API — Core Classes



3.3 Wallet.API — Consumer Pattern



4. Solution Structure

The Visual Studio solution file (WalletPlatform.slnx) organises all projects in a flat structure grouped by folder.

4.1 Standard Folder Structure Per Service

```
ServiceName.API/
├── Controllers/        ← HTTP endpoints only — no business logic
├── Data/
│   ├── ServiceDbContext.cs  ← EF Core DbContext with OnModelCreating
│   └── Migrations/        ← Auto-generated EF Core migrations
├── DTOs/
│   ├── Request/           ← What comes IN from clients
│   └── Response/          ← What goes OUT to clients
├── Entities/           ← Database table mappings
├── Enums/              ← Domain enumerations stored as strings
├── Events/
│   ├── Publishers/        ← Publishes events to RabbitMQ
│   └── Consumers/         ← BackgroundService RabbitMQ consumers
├── Middleware/         ← ExceptionMiddleware (all services)
├── Repositories/
│   ├── Interfaces/        ← IRepository contracts for DI
│   └── Implementations/   ← Concrete EF Core implementations
├── Services/
│   ├── Interfaces/        ← IService contracts for DI
│   └── Implementations/   ← Business logic — no HTTP, no SQL
├── Validators/         ← FluentValidation rules (Auth service)
├── appsettings.json    ← Connection strings, JWT, RabbitMQ config
├── appsettings.Development.json
└── Program.cs          ← DI registration, middleware pipeline
```

5. Interface Contracts

5.1 Key Interfaces

Interface	Purpose
IAuthService	RegisterAsync, LoginAsync, RefreshTokenAsync, LogoutAsync, GetProfileAsync
ITokenService	GenerateAccessToken, GenerateRefreshToken, GetUserIdFromToken
IKYCService	SubmitAsync, GetStatusAsync, ApproveAsync, RejectAsync, GetPendingAsync
IUserRepository	CRUD for User entity + RefreshToken + AuditLog operations
IKYCRepository	CRUD for KYCRecord — GetByUserId, GetPending, Update
IWalletService	GetWallet, CreateWallet, Fund, Withdraw, Freeze, Unfreeze, Debit, Credit
IWalletRepository	CRUD for Wallet + PaymentMethod + FreezeLog
ITransactionService	InitiatePayment, GetTransaction, GetHistory, HandleFailed
ILedgerService	GetAccountLedger, GetDerivedBalance
ITransactionRepository	CRUD + idempotency record management
ILedgerRepository	Append-only CreateAsync/CreateManyAsync + GetAccountBalance
IRewardsService	GetAccount, GetHistory, GetTiers, CreateAccount, AwardPoints, DeductPoints
IPointRuleService	GetAll, Create, Deactivate
ILoyaltyRepository	CRUD + GetTierForPoints + GetNextTier
IPointRuleRepository	GetActiveRule, GetAll, Create, Deactivate
ICatalogService	GetAll, GetById, Create, Deactivate
IRedemptionService	Redeem, GetUserRedemptions, GetById, Complete, Fail
INotificationService	SendAsync, GetUserNotifications
IEmailService	SendAsync — abstraction for email provider swap (SendGrid etc.)
IRabbitMQPublisher	Publish<T>(message, queueName) — shared library contract

6. Complete API Endpoint Catalogue

6.1 Auth.API (:5010)

Method	Path	Auth	Description
POST	/api/auth/register	Public	Register new user — returns JWT tokens immediately
POST	/api/auth/login	Public	Login — returns access + refresh tokens
POST	/api/auth/refresh-token	Public	Exchange refresh token for new access token
POST	/api/auth/logout	JWT	Revoke refresh token
GET	/api/auth/profile	JWT	Get current user profile
POST	/api/kyc/submit	JWT	Submit KYC document for review
GET	/api/kyc/status	JWT	Get current user KYC status
GET	/api/kyc/pending	Admin	List all submitted KYC records awaiting review
POST	/api/kyc/{id}/approve	Admin	Approve KYC — triggers wallet activation event
POST	/api/kyc/{id}/reject	Admin	Reject KYC with mandatory reason

6.2 Wallet.API (:5020)

Method	Path	Auth	Description
GET	/api/wallet	JWT	Get current user wallet with payment methods
POST	/api/wallet/fund	JWT	Add money to wallet — publishes WalletFundedEvent
POST	/api/wallet/withdraw	JWT	Withdraw from wallet to payment method
POST	/api/wallet/{id}/freeze	Admin	Freeze wallet with mandatory reason
POST	/api/wallet/{id}/unfreeze	Admin	Unfreeze wallet with mandatory reason

6.3 Transaction.API (:5030)

Method	Path	Auth	Description
POST	/api/transactions/pay	JWT	Initiate payment — requires Idempotency-Key header
GET	/api/transactions/{id}	JWT	Get single transaction by ID
GET	/api/transactions/history	JWT	Paginated transaction history (page, pageSize)
GET	/api/transactions/ledger	JWT	Paginated ledger entries for current user

Method	Path	Auth	Description
GET	/api/transactions/ledger/balance	JWT	Ledger-derived balance — audit check vs wallet balance

6.4 Rewards.API (:5040) · Catalog.API (:5050) · Notification.API (:5080)

Method	Path	Auth	Description
GET	/api/rewards	JWT	Get loyalty account, tier, points balance
GET	/api/rewards/history	JWT	Paginated point transaction history
GET	/api/rewards/tiers	JWT	List all reward tiers with thresholds
GET	/api/admin/rewards/rules	Admin	List all point rules
POST	/api/admin/rewards/rules	Admin	Create new point rule
DELETE	/api/admin/rewards/rules/{id}	Admin	Deactivate a point rule
GET	/api/catalog	Public	Browse catalog items — no auth required
GET	/api/catalog/{id}	Public	Get single catalog item
POST	/api/catalog	Admin	Create new catalog item
DELETE	/api/catalog/{id}	Admin	Deactivate catalog item
POST	/api/redemption	JWT	Redeem points for catalog item
GET	/api/redemption/my	JWT	Get current user redemption history
GET	/api/redemption/{id}	JWT	Get single redemption by ID
GET	/api/notification	JWT	Get notification delivery logs

7. Error Handling Specification

7.1 HTTP Status Code Mapping

Exception Type	HTTP Status	Example Scenario
InvalidOperationException	400 Bad Request	Wallet frozen, insufficient balance, duplicate email
UnauthorizedAccessException	401 Unauthorized	Invalid password, expired token, missing token
KeyNotFoundException	404 Not Found	User not found, wallet not found, KYC not found
Any other Exception	500 Internal Server Error	Database timeout, null reference, unhandled bug
FluentValidation failure	400 Bad Request	Invalid email format, weak password, missing field
Rate limit exceeded	429 Too Many Requests	More than 5 registrations per minute from same IP

7.2 Standard Error Response Format

```
{
 "success": false,
 "message": "Wallet is frozen. Please contact support.",
 "data":    null,
 "errors":  []
}
```

All error responses follow the ApiResponse<T> standard defined in WalletPlatform.Shared. The message field is safe to display to end users for known exceptions. For 500 errors, the message is always 'An unexpected error occurred.' — the real error is only in server logs.

8. Dependency Injection Registration

Service	Lifetime	Reason for lifetime choice
IAuthService → AuthService	Scoped	One per HTTP request — shares DbContext with repository
ITokenService → TokenService	Scoped	One per request — reads config, no state
IKYCService → KYCService	Scoped	One per request — shares DbContext
IUserRepository → UserRepository	Scoped	One per request — EF Core DbContext is scoped
IKYCRepository → KYCRepository	Scoped	One per request — EF Core DbContext is scoped
IRabbitMQPublisher → RabbitMQPublisher	Singleton	One TCP connection shared — expensive to create per request
AuthEventPublisher	Singleton	Wraps Singleton publisher — must match lifetime
UserRegisteredConsumer	Singleton	BackgroundService — runs for app lifetime
KYCApprovedConsumer	Singleton	BackgroundService — runs for app lifetime
AuthDbContext	Scoped	EF Core recommendation — one context per request

9. Configuration Reference

9.1 appsettings.json — All Services

```
// All services share identical JWT and RabbitMQ sections
"ConnectionStrings": {
  "DefaultConnection": "Data Source=SERVER\SQLEXPRESS;Initial Catalog=SERVICE_db;
                        Integrated Security=True;TrustServerCertificate=True;"
},
"Jwt": {
  "Key":          "WalletPlatform_SuperSecret_JWT_Key_2024_MustBe32CharsMin!!",
  "Issuer":       "WalletPlatform.Auth",
  "Audience":      "WalletPlatform.Client",
  "ExpiryMinutes": "60"
},
"RabbitMQ": {
  "HostName": "localhost",
  "UserName":  "guest",
  "Password":  "guest"
}
```

9.2 Port Assignments

Service	HTTP Port	HTTPS Port
ApiGateway	5000	5001
Auth.API	5010	5011
Wallet.API	5020	5021
Transaction.API	5030	5031
Rewards.API	5040	5041
Catalog.API	5050	5051
Notification.API	5080	5081
Reporting.API	5070	5071
RabbitMQ AMQP	5672	—
RabbitMQ Management	15672	—
Angular Dev Server	4200	—

10. Document Control

Document Title	WalletPlatform Low Level Design Document
Version	1.0
Status	Final
Sections	Flow diagrams, DB schema, class diagrams, solution structure, API catalogue, DI map
Services Covered	Auth, Wallet, Transaction, Rewards, Catalog, Notification, Gateway + Shared

End of Document